

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Практическое занятие -6-7***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Практическая работа - 6-7
Построение диаграммы Вариантов использования
(диаграммы прецедентов use case diagram)

Цель:

Изучение программного инструмента объектно-ориентированного моделирования StarUML ™, использование функциональных возможностей для создания диаграммы прецедентов.

Задачи:

1. *Настройка среды в соответствии с требованиями пользователя*
2. *Моделирование по заданным условиям предметной области. Создание заданной диаграммы прецедентов*
3. *Выполнение индивидуального задания*
4. *Отчет*

Ход работы

Задание 1. Настройка среды

Основная структурная единица в StarUML – это проект. Проект сохраняется в одном файле в формате XML с расширением «.UML».

Проект может содержать одну или несколько моделей и различные представления этих моделей (View) – визуальные выражения информации, содержащейся в моделях. Каждое представление модели содержит диаграммы – визуальные образы, отображающие определенные аспекты модели.

Новый проект будет автоматически создан при запуске программы StarUML. При этом вам будет предложено в диалоговом окне выбрать один из подходов (Approaches), поддерживаемых StarUML (см. рис. 1).

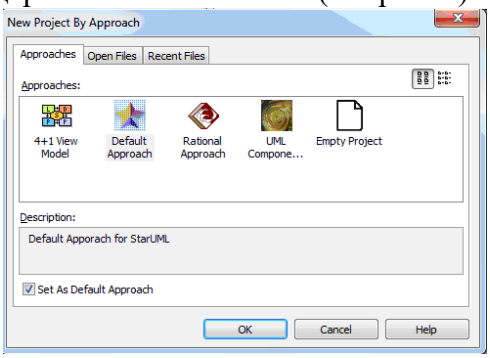


Рисунок 1. Выбор подхода

После выбора одного из предложенных подходов, появится основное окно программы (рис.2).

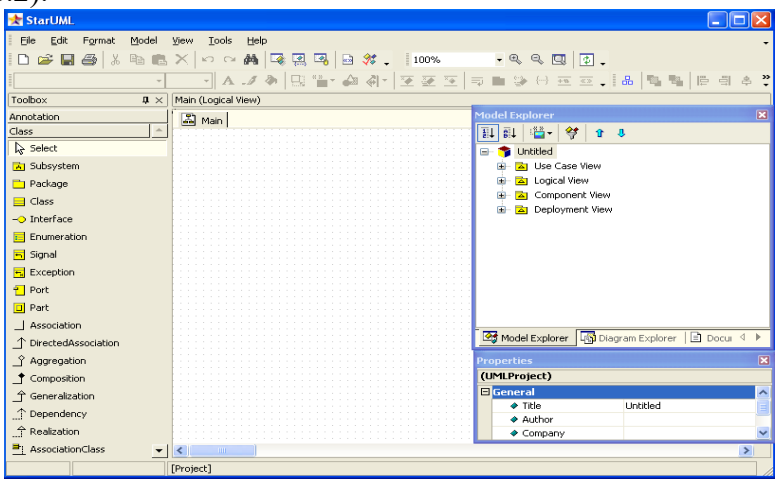


Рисунок 2. Основное окно программы

В верхней части окна расположено главное меню, кнопки быстрого доступа. Слева расположена панель элементов (Toolbox) с изображениями элементов диаграммы. Элементы соответствуют типу выбранной диаграммы. В центре находится рабочее поле диаграммы, на котором она может быть построена с использованием соответствующих элементов панели инструментов.

Справа находится **инспектор модели**, на котором можно найти вкладки навигатора модели Model Exploer, навигатора диаграмм Diagram Exploer, окно редактора свойств Properties, окно документирования элементов модели Documentation и редактор вложений Attachments. Внешний вид инспектора модели с вкладками представлен ниже (рис. 3).

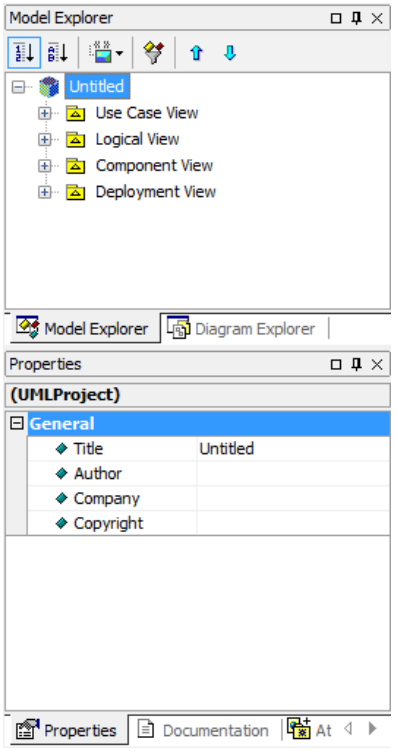


Рисунок 3. Инспектор модели

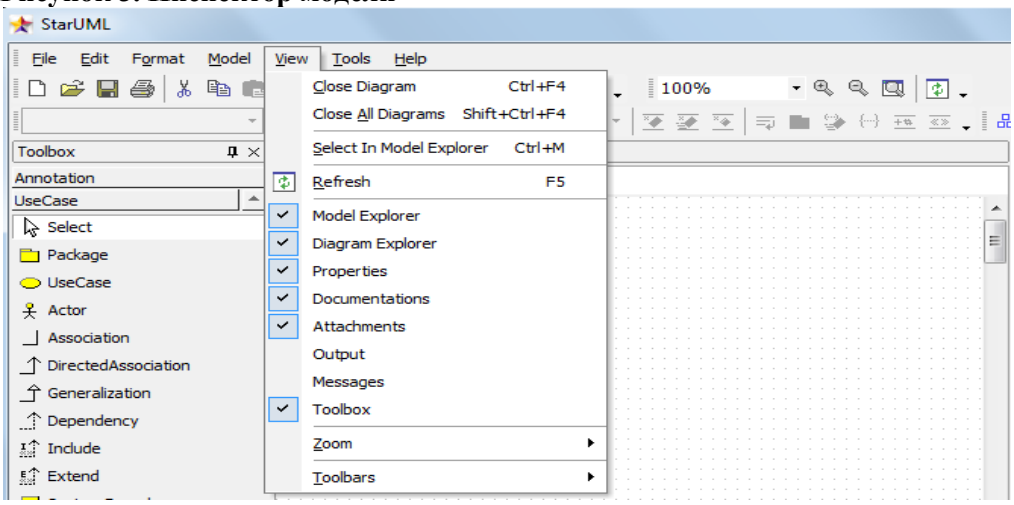


Рисунок 4. Пункт меню View. Управление видом инспектора модели

Управлять видом инспектора модели, панели элементов, закрывать и открывать редакторы инспектора можно с помощью пункта меню View (рис.4).

Иерархическая структура проекта отображается справа на навигаторе модели (Model Exploer). В зависимости от выбранного подхода на навигаторе модели будут отображены различные пакеты представлений модели. Каждый пакет представления будет содержать элементы созданных моделей и диаграмм.

Если при создании нового проекта моделирования необходимо выбрать подход Rational Approach.

При таком подходе в навигаторе будут присутствовать четыре пакета представлений модели системы (см. рис. 5):

- Use Case View – представление требований к системе, описывает, что система должна делать;
- Logical View – логическое представление системы, описывает, как система должна быть построена;
- Component View – представление реализации, описывает зависимость между программными компонентами;
- Deployment View – представление развертывания, описывает аппаратные элементы, устройства и программные компоненты.

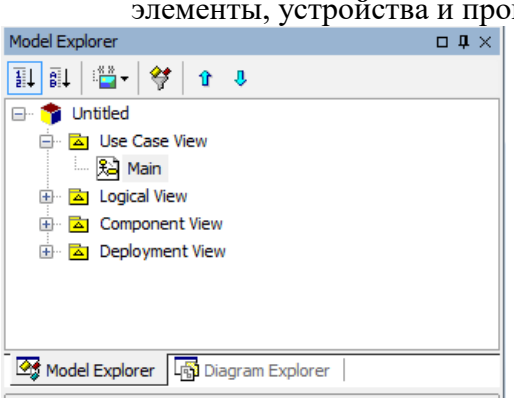


Рисунок 5. Навигатор модели

Каждое представление содержит одну диаграмму с именем Main. Если щелкнуть по ней два раза, то откроется рабочее поле этой диаграммы и соответствующая панель инструментов.

Пример. Если щелкнуть два раза по диаграмме Main представления Use Case View, то откроется рабочее этой диаграммы и ее панель элементов (рис. 6).

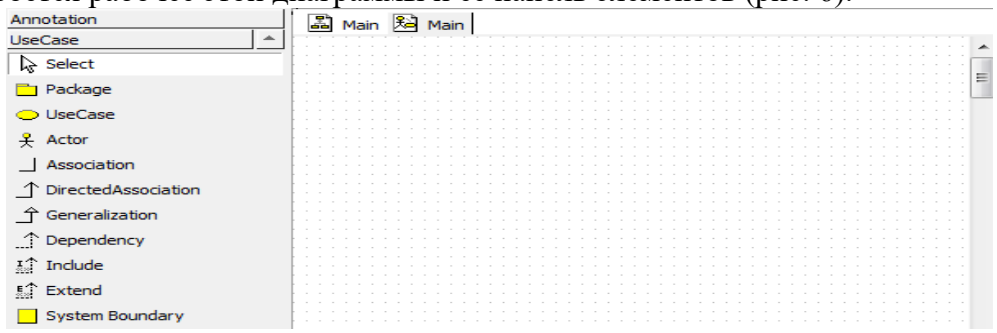


Рисунок 6. Рабочее поле диаграммы прецедентов Main и ее панель элементов

Ниже иерархии представлений отображаются свойства выделенного элемента модели или диаграммы (в данном случае свойства диаграммы Main, так как она выделена в навигаторе модели) (см. рис. 7).

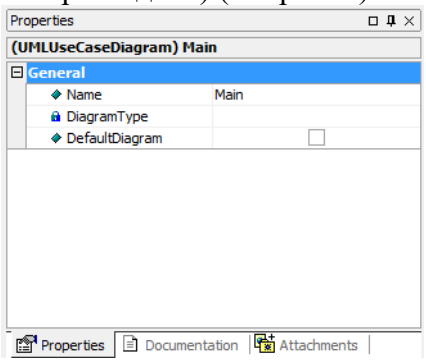


Рисунок 7. Редактор свойств

Задание 2 Моделирование программного продукта по заданным условиям предметной области

2.1. Создайте новый проект в StarUML, выбрав **Rational Approach** из списка предложенных подходов. Модель будет иметь четыре представления: **Use Case, Logical, Component и Deployment**. Данный подход по структуре представлений наиболее соответствует методологии Rational Unified Procces (RUP), которая поддерживает итеративный процесс разработки информационных систем.

Сохраните созданный вами проект под любым именем.

2.2.Выполните визуальное моделирование систем с использованием UML на примере проектирования системы заказов **интернет-магазина «Style»**.

Описание предметной области

Каждый товар в каталоге описывается артикулом, размерным рядом, ценой и фото с кратким описанием.

Покупатель может загрузить каталог товаров. Каталог не содержит разделы, имеет блочную структуру, состоит из набора товаров с фото, ценой и размерами. Покупатель складывает понравившиеся товары в корзину, при этом выбирая размер и количество необходимого товара данного артикула.

Корзину можно изменить: просмотреть, удалить товар, изменить количество позиций одного артикула, вернуться в каталог.

Когда покупатель делает заказ, он вводит свои личные данные, телефон и оплачивает его по банковской карте (если заказ не оплачен, то он и не сделан).

После того как сделан заказ, его можно забрать со склада через 1 рабочий день. Данные о заказе поступают сотруднику магазина, назовем его сотрудником отдела продаж, он проверяет наличие товаров и передает его кладовщику на комплектацию. Кладовщик, собрав заказ, делает отметку о готовности.

Заказ выдается со склада кладовщиком. Кладовщик выдает заказ и отмечает в системе, что заказ выдан.

Магазин не занимается доставкой заказов, не делает скидок. Для того чтобы ограничить масштаб задачи, мы не рассматриваем систему снабжения магазина новыми товарами. Этим занимается другая система, назовем ее Склад. Информация о проданных товарах (т.е. сделанных заказах) поступает также в систему Склад.

Поскольку на протяжении от создания до выдачи заказа, он проходит разные стадии, то будет разумно ввести понятие статуса заказа. Сотрудники магазина могут статус заказа изменять, а покупатель может проследить за сборкой заказа. В таком случае наша система предоставляет еще одну функцию: узнать статус заказа.

***2.3. Изучите представленную информацию*:**

Поведение системы (т.е. функциональность, которую она обеспечивает) описывают с помощью функциональной модели, которая отображает системные прецеденты (use cases, случаи использования), системное окружение (действующих лиц, актеров, actors) и связи между ними (use cases diagrams).

Диаграмма вариантов использования (диаграмма прецедентов, use case diagram) — это диаграмма, на которой изображаются отношения между актерами и вариантами использования.

С помощью этой диаграммы можно:

- Определить общие **границы и контекст** моделируемой предметной области на начальных этапах проектирования системы;
- Сформулировать общие **требования** к функциональному поведению проектируемой системы;

- Разработать исходную **концептуальную модель** системы для ее последующей детализации в форме логических и физических моделей;
- Подготовить **исходную документацию** для взаимодействия разработчиков системы с ее заказчиками и пользователями.

Диаграмма вариантов использования (прецедентов) представляет собой граф, в вершинах которого расположены актеры или прецеденты, связи между вершинами – это разного вида отношения.

Актером (действующее лицо, actor) называется любой объект, субъект или система, взаимодействующая с моделируемой системой извне.

Это может быть человек, техническое устройство, программа или любая другая система, которая служит источником воздействия на моделируемую систему так, как определит разработчик. На диаграммах вариантов использования актер изображается в виде человечка, под которым записывается его имя (рис. 8).

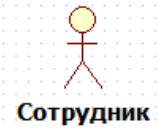


Рисунок 8. Действующее лицо (актер)

Вариант использования (прецедент, use case) — описание множества последовательности действий (включая варианты), выполняемых системой для того, чтобы актер мог получить определенный результат.

Каждый вариант использования должен быть независимым в том смысле, что если он всегда выполняется совместно с другим вариантом использования, то, скорее всего, это один прецедент, а не два, либо они связаны отношением включения или расширения, о чем речь пойдет позже. Как следует из определения, прецедент (или вариант использования) должен обладать результирующей ценностью для актера, актер должен получить некоторый результат исполнения прецедента. Скорее всего, после исполнения прецедента в системе произойдут некоторые изменения: появятся новые данные, изменится поведение. Каждый вариант использования должен исполняться от начала до конца.

Прецедент описывает, *что делает* система, но никак *не уточняет, как* она это делает. Заметим, что диаграмма прецедентов не отображает последовательность, в которой будут выполняться варианты использования. На диаграмме прецедент изображается в виде эллипса. Имя прецедента может состоять из нескольких слов и знаков препинания (за исключением двоеточия), как правило, имя выбирают в виде словосочетания или глагольного выражения (рис. 9).



Рисунок 9. Варианты использования (прецеденты)

Одним из наиболее важных (и дорогостоящих) этапов проектирования информационных систем является этап определения требований к системе. Если требования заказчика информационной системы разработчиками будут определены не корректно, то в итоге заказчик может получить совсем не ту систему, которую он ожидал.

Моделирование прецедентов и актеров помогает лучше понять требования, предъявляемые к системе, и согласовать их с заказчиком с помощью демонстрации и обсуждения диаграммы прецедентов. Прецеденты и актеры – это отражение требований к системе, они показывают, кто и для чего будет использовать будущую систему.

Пример. Определить актеров и прецеденты системы заказов магазина «Style».

Покупатель делает заказ, складывая товары в корзину. Возможна только одна форма оплаты: банковской картой по интернету, невозможно оформление заказа без оплаты. Заказ имеет статус: *оплачен, передан на комплектацию, собран, получен*. Статус заказа изменяется автоматически либо сотрудником магазина. Покупатель может узнать статус своего заказа по уникальному номеру заказа.

Система не занимается поставками товаров в магазин. Этим занимается другая система, назовем ее **Склад**.

Таким образом, с нашей системой взаимодействуют *покупатель, сотрудники магазина* и внешняя система **Склад**. С нашей системой взаимодействуют *сотрудник отдела продаж*, который проверяет оплату заказа и отправляет его на комплектацию, и *кладовщик*, который собирает заказ и выдает его покупателю. С точки зрения бизнеса – это две разных должности, выполняющих разные функции, но с точки зрения системы они играют одну роль сотрудника, изменяющего статус заказа покупателя с использованием программного обеспечения моделируемой системы. В этом смысле для системы нет разницы между сотрудником отдела продаж и кладовщиком. Выбирая действующих лиц, нужно помнить о том, что мы должны отразить **их роль, а не должность**. Введем обобщающее сотрудников действующее лицо – **Сотрудник**. Другой пример: сотрудник магазина «Style» (положим, кладовщик) может выступать в роли сотрудника и общаться с системой как сотрудник магазина, а может выступать и в роли покупателя, сделав заказ в магазине. Не смотря на то, что физически это один человек, он выступает в роли двух актеров: покупателя и сотрудника.

Итак, ***актеры*** системы заказов магазина «Style» будут следующие:

Покупатель, Сотрудник, Система Склад.

Покупатель использует нашу систему для того, чтобы заказать вещи, он просматривает каталог, добавляет понравившиеся ему товары в корзину, открывает корзину, удаляет из нее товары или изменяет их количество и, наконец, может оформить свой заказ, при этом его оплатив. В конечном итоге результат использования системы покупателем будет получен, если он выполнил все эти действия от начала до конца. Поэтому не будем разделять заказ товаров на несколько прецедентов, а выделим только один: Заказ товаров.

Покупатель, сделав заказ в магазине «Style», может в дальнейшем узнавать статус своего заказа, это тоже случай использования системы, назовем его Получение информации о заказе.

Сотрудник должен изменять статус сделанного заказа, для него определим прецедент Управление статусом заказа.

Система Склад должна получать информацию о сделанных заказах для возможности управления наличием товаров на складе, для нее также должен быть доступен прецедент Получение информации о заказе.

Итак, ***прецеденты*** системы заказов магазина «Style»: ***Заказ товаров, Управление статусом заказа, Получение информации о заказе.***

Отношения между прецедентами и актерами

Связи и взаимоотношения, существующие между элементами модели, в UML описываются с помощью отношений, изображаемых на диаграммах.

Отношение (relationship) **—** это семантическая связь между отдельными элементами модели.

Между актерами и прецедентами диаграммы вариантов использования могут существовать разного рода отношения, показывающие, что экземпляры действующих лиц и вариантов использования взаимодействуют друг с другом. Действующие лица могут взаимодействовать с несколькими прецедентами и между собой, равно как и прецеденты могут быть связаны между собой особого типа отношениями.

В основном на диаграммах прецедентов используются следующие типы отношений:

- *ассоциации (association relationship);*
- *включения (include relationship);*
- *расширения (extend relationship);*
- *обобщения (generalization relationship).*

Ассоциация – это структурное отношение, показывающее, что объект неким образом связан с другим объектом.

Отношение этого типа используется не только на диаграммах прецедентов, но и на других диаграммах. Если между элементами модели показано отношение ассоциации, то это означает, что между ними существует семантическая связь. Ассоциативное отношение может быть направленным. В этом случае направление связи показывает, кто является инициатором. Если отношение направленно от актера к прецеденту, то это означает, что актер инициирует выполнение прецедента.

Пример. Покупатель в системе заказов магазина «Style» инициирует выполнение прецедента Заказ товаров (см. рис. 10).

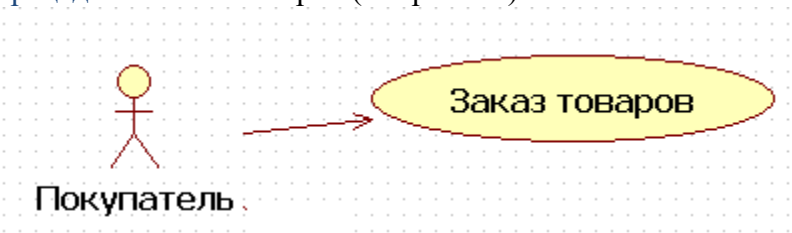


Рисунок 10. Отношение ассоциации между актером и прецедентом

Между прецедентами также возможны взаимоотношения, которые описываются отношениями двух типов: включения и расширения (дополнения).

Включение (include) говорит о том, что исходный прецедент явным образом включает в себя поведение целевого.

Другими словами, включение создается, когда один прецедент использует другой. При этом исполнение базового прецедента невозможно без исполнения используемого. Изображается включение в виде пунктирной стрелки с надписью <<include>>, которая направлена от базового элемента к используемому.

Пример. В системе заказов магазина «Style» невозможен заказ товаров без оплаты. На диаграмме прецедентов это можно отразить так, как это показано на рисунке 11

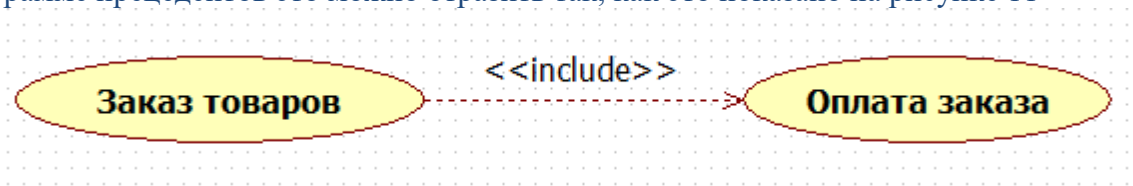


Рисунок 11. Отношение включения между прецедентами

Расширение (extend) показывает, что целевой прецедент расширяет поведение исходного.

Используемый прецедент выполняется не всегда вместе с базовым, а только при выполнении дополнительных условий, таким образом, расширяя функциональность базового элемента. Изображается расширение пунктирной стрелкой с надписью <<extend>>, направленной от используемого варианта использования к базовому.

Пример. При заказе товаров в системе заказов магазина «Style» покупатель может изменить содержание корзины перед тем, как оформить заказ окончательно, а может оставить корзину без изменений. Изменение корзины – это опция, которую на диаграмме вариантов использования мы можем изобразить с помощью расширения (рис. 12).

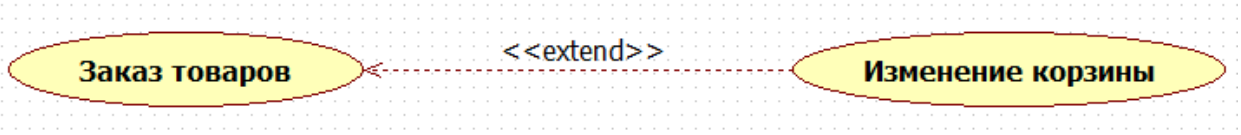


Рисунок 12. Отношение расширения между прецедентами

Обозначения отношений <<include>> и <<extend>> есть не что иное, как обозначения стереотипов, которые широко используются в UML для создания новых элементов модели путем расширения функциональности базовых элементов.

Стереотип (Stereotype) – это механизм, позволяющий категоризировать элементы модели.

С помощью стереотипов можно создавать своего рода подтипы типов. Это позволяет UML иметь минимальный набор элементов, которые могут быть дополнены при необходимости для создания связующих базовых элементов в системе. В UML стереотип обозначается именем, которое записывается в двойных угловых скобках: **<<имя стереотипа>>.**

В UML можно создавать собственные стереотипы на основе уже имеющихся типов, но также существуют и стандартные, заранее определенные стереотипы нотации UML. Так, отношение зависимости расширяется для прецедентов и актеров с помощью двух стереотипов <<include>> и <<extend>>.

Ассоциация – это коммуникативное отношение, которое соответствует стереотипу <<communicate>>, который, впрочем, всегда опускается.

Два и более актера могут иметь общие свойства, т.е. взаимодействовать с одним и тем же множеством вариантов использования одинаковым образом. Такая общность свойств и поведения представляется в виде отношения обобщения с другим, возможно, абстрактным актером, который моделирует соответствующую общность ролей.

Обобщение (Generalization) – это отношение между общей сущностью и ее конкретным воплощением.

На диаграммах обобщение обозначается стрелкой с не закрашенным треугольником на конце, направленной от частного элемента к общему.

Пример. Для изменения статуса заказов в магазине «Style» с проектируемой системой будут работать сотрудник отдела продаж и кладовщик. На диаграмме мы можем показать с помощью отношения обобщения взаимосвязь между актером Сотрудник и актерами Сотрудник отдела продаж и Кладовщик (рис. 13).

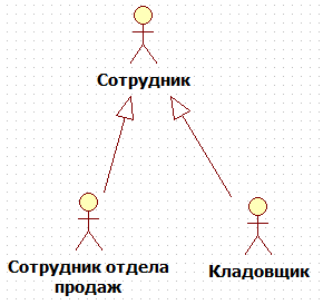


Рисунок 13. Отношение обобщения между актерами

Актеры, прецеденты и отношения – это основные элементы нотации диаграмм вариантов использования. Диаграмма вариантов использования помогает отобразить основные требования к моделируемой системе и обеспечить взаимопонимание функциональности системы между разработчиком и заказчиком. Можно построить одну, главную диаграмму прецедентов, на которой будут отражены границы системы (актеры) и ее основная функциональность (прецеденты). Для более подробного представления системы допускается построение вспомогательных диаграмм прецедентов.

Создание диаграммы вариантов использования (прецедентов)

Построение диаграммы прецедентов в StarUML

В StarUML главная диаграмма прецедентов называется Main и располагается в представлении Use Case. Если в навигаторе модели щелкнуть два раза по имени этой диаграммы, то откроется ее рабочее поле. Для того чтобы создать прецедент, щелкните по овальному символу прецедента на панели элементов слева от рабочего поля диаграммы, а затем щелкните по тому месту на рабочем поле диаграммы, в которое вы хотите поместить прецедент. Аналогичным образом создается актер. Когда элемент помещается на поле диаграммы, он становится доступен для редактирования имени и некоторых свойств.

В выделенное **поле введите новое имя прецедента** или актера (рис. 14).

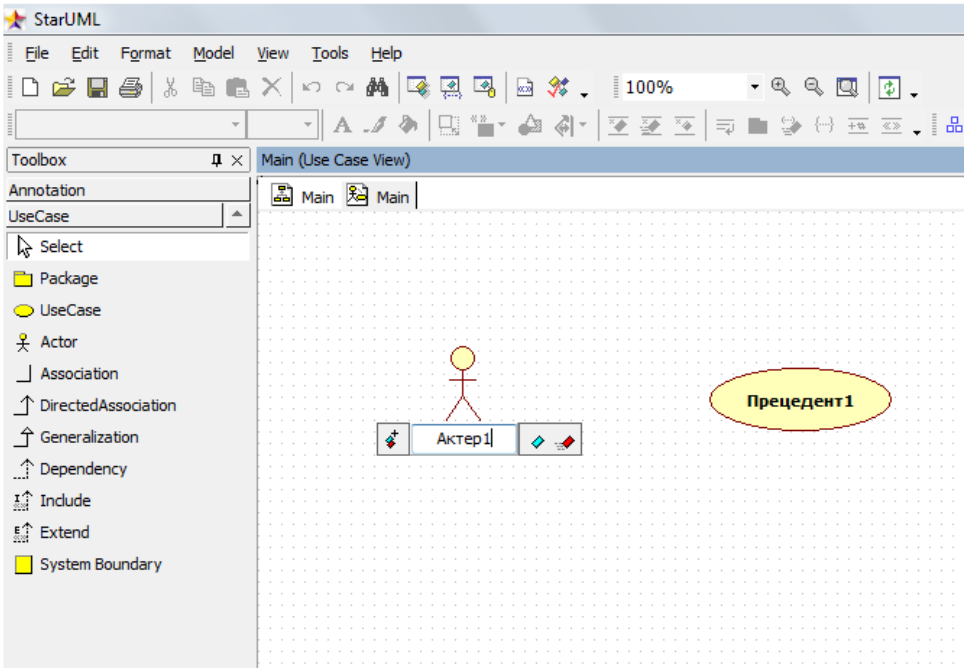


Рисунок 14. Именование элементов в StarUML

Для создания отношения между элементами диаграммы щелкните по изображению соответствующего отношения на панели элементов справа, а затем проведите линию от одного элемента к другому, удерживая левую кнопку мыши (рис. 15).

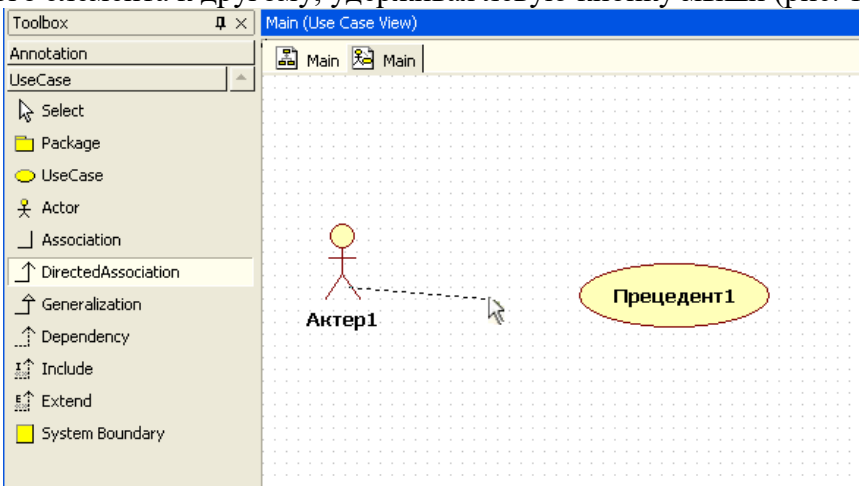


Рисунок 15. Создание отношений между элементами в StarUML

Чтобы удалить элемент с диаграммы достаточно щелкнуть левой кнопкой мыши по этому элементу, а затем нажать кнопку Delete, либо щелкнуть правой кнопкой мыши по элементу и в контекстном меню выбрать Edit -> Delete (рис. 16).

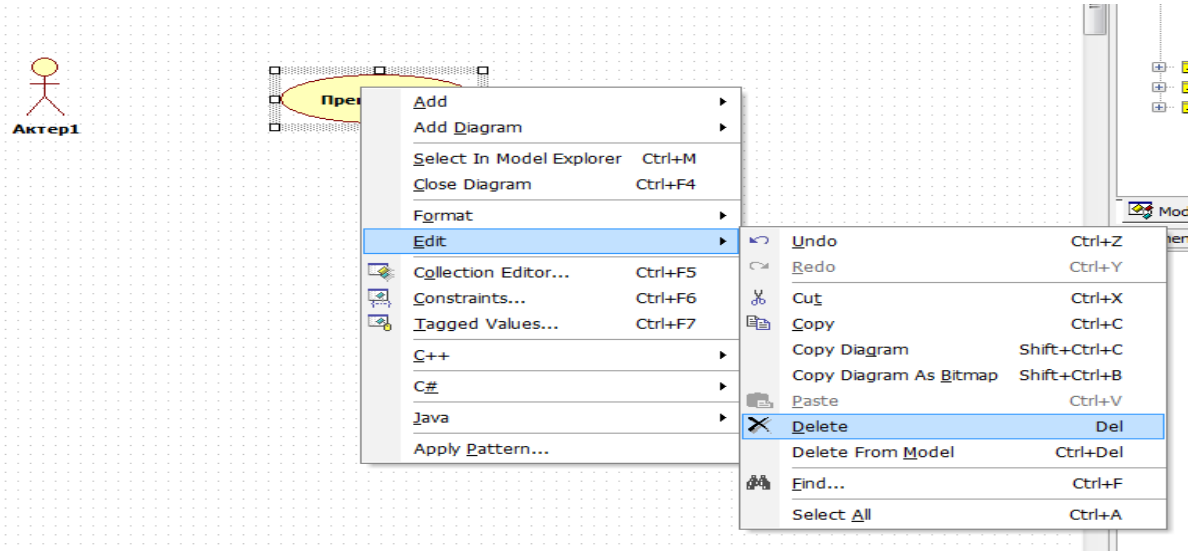


Рисунок 16. Удаление элемента диаграммы в StarUML

Обратите внимание, что элемент был удален с диаграммы, но не из модели (рис. 17)! Его можно найти в навигаторе модели, не смотря на то, что на диаграмме он больше не отображается (элемент Прецедент1):

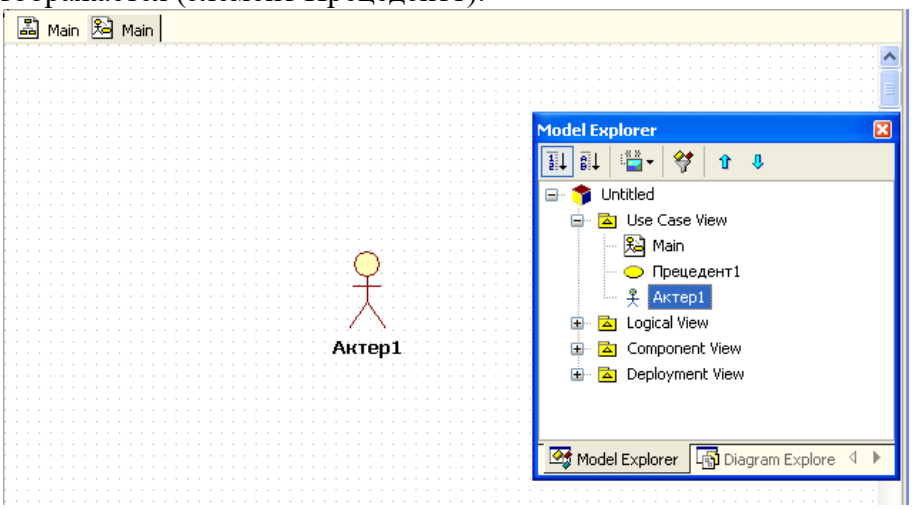


Рисунок 17. Элемент Прецедент1 удален с поля диаграммы, но отображается в навигаторе модели

Для того, чтобы вернуть элемент на диаграмму, нужно перетащть его с навигатора модели на поле диаграммы.

Для того чтобы **удалить элемент из модели** нужно щелкнуть по нему на диаграмме или по его изображению в навигаторе модели правой кнопкой мыши и в контекстном меню выбрать пункт Delete from Model. Элемент будет полностью удален.

Задание 2.4. Для системы заказов магазина «Style» определили ***актеров Покупатель, Сотрудник, Система Склад и прецеденты Заказ товаров, Управление статусом заказа, Получение информации о заказе***.

Постройте основную диаграмму прецедентов (рис. 18).

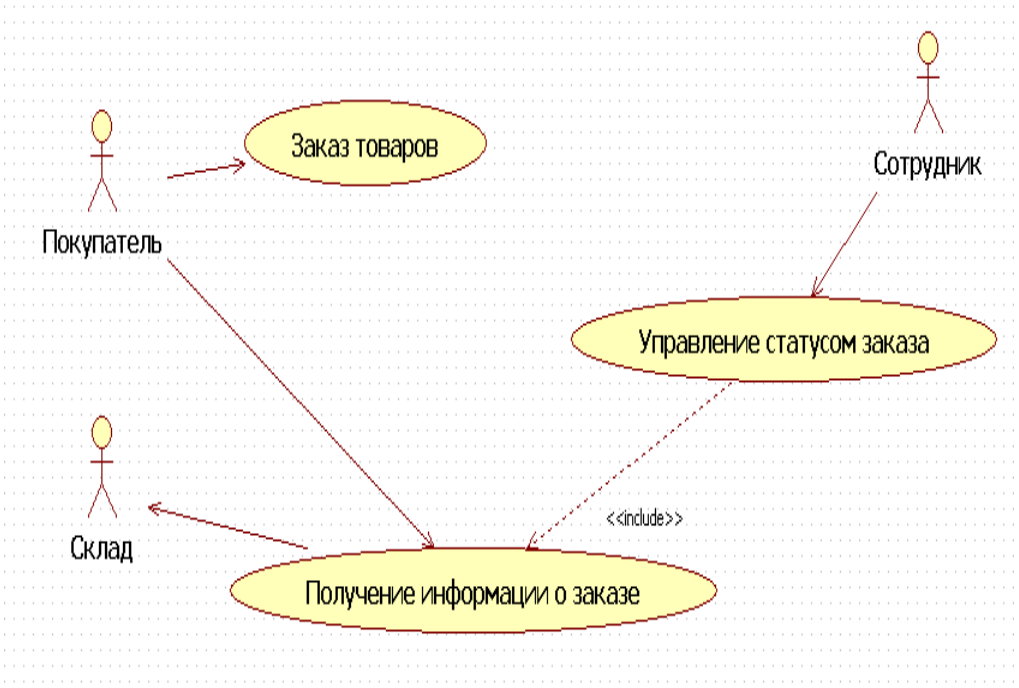


Рисунок 18. Основная диаграмма вариантов использования системы заказов магазина «Style»

Для актера Покупатель и прецедента Заказ товаров установили отношение направленной ассоциации: Заказ товаров инициализируется Покупателем. Сотрудник имеет возможность управлять статусом заказа, при этом он непременно участвует в прецеденте Получение информации о заказе. Направленную ассоциацию от Получение информации о заказе к актеру Система Склад можно понимать как автоматическую передачу данных из моделируемой системы в систему снабжения товарами Склад.

В модель нужно включить краткое описание каждого актера или прецедента, делается это для того, чтобы между разработчиком и заказчиком системы не оставалось «белых пятен» и расхождений в понимании функциональности системы и ролей взаимодействующих с ней актеров. Для каждого актера описывается роль, которую он играет в системе, а для каждого прецедента – его назначение и функциональность. Также можно уточнить, каким актером запускается прецедент.

Задание 2.5. ***Выполните документирование элементов модели в StarUML***

В StarUML добавление описания к элементам модели делается следующим образом. Выделите элемент модели, щелкнув по нему мышкой, и откройте редактор Documentation. Если он не отображается справа на одной из вкладок инспектора модели, то откройте его, используя меню View → Documentation. Напротив пункта Documentation должна стоять «галочка».

Введите описание элемента в окно документирования (рис. 19).

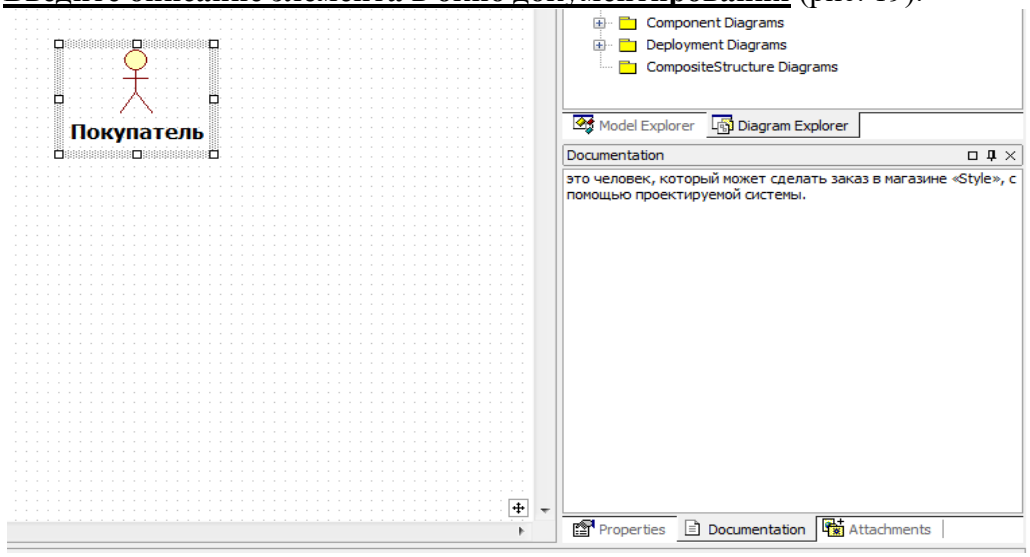


Рисунок 19. Документирование элемента модели в StarUML

Все элементы модели должны быть **задокументированы**. Описанный выше способ подходит для любого элемента любой диаграммы.

Для актеров и прецедентов системы заказов магазина «Style» сделайте краткое описание.

Покупатель – это человек, который может сделать заказ в магазине «Style», с помощью проектируемой системы.

Сотрудник – это все сотрудники магазина «Style», которые могут получать информацию о сделанных заказах и изменять статус заказа в системе в зависимости от того шага, на котором находится обработка данного заказа.

Система Склад – это внешняя система, которая получает информацию о сделанных в магазине «Style» заказах для того, чтобы обеспечить учет наличия товаров на складе и снабжение товарами.

Заказ товаров – этот прецедент запускается покупателем для того, чтобы оформить заказ в магазине «Style». Состоит из просмотра каталога, добавления товаров в корзину, просмотра корзины, изменения содержания корзины и оформления заказа, включая оплату.

Управление статусом заказа – этот прецедент используется сотрудниками магазина для изменения статуса заказа в процессе его обработки. Получение информации о заказе – прецедент используется всеми актерами для просмотра информации о заказе.

Для того чтобы создать еще одну диаграмму (любого типа), например, детализирующую прецедент, щелкните правой кнопкой мыши по папке Use Case Model и в появившемся контекстном меню выберите Add Diagram, затем выберите из списка диаграмму, которую вы хотите добавить. Создать дополнительную диаграмму прецедентов можно, выбрав пункт Use Case Diagram (рис. 20).

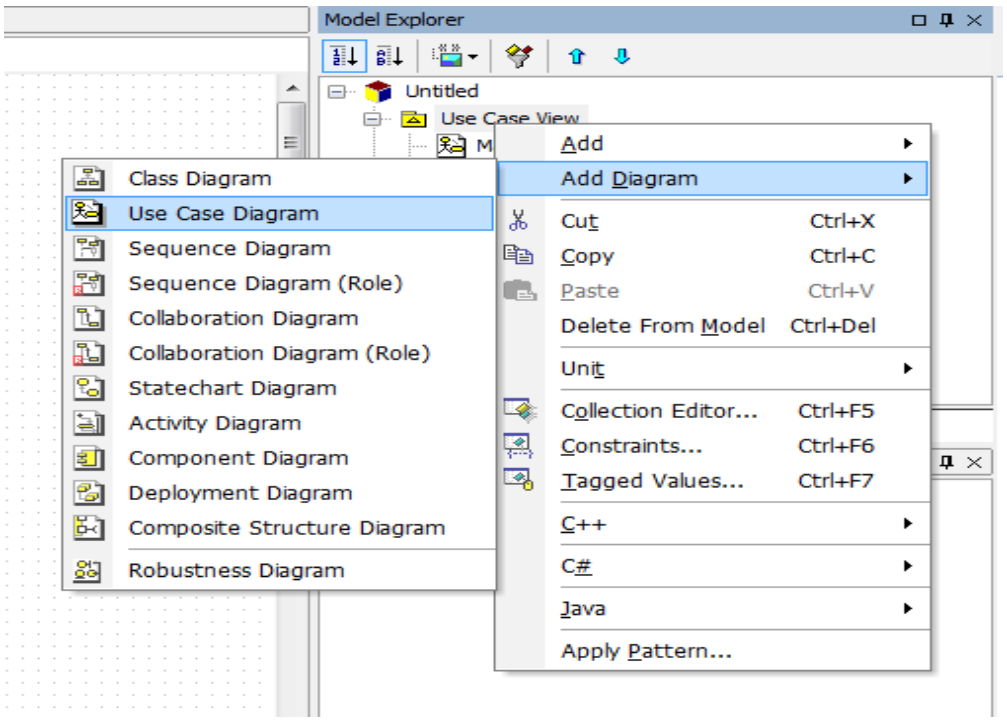


Рисунок 20. Создание дополнительной диаграммы прецедентов

Задание 2.5. Наиболее значимым для данной системы и ее актеров прецедентом является прецедент Заказ товаров. ***Постройте для него дополнительную диаграмму прецедентов***, поясняющую этот вариант использования (рис. 21).

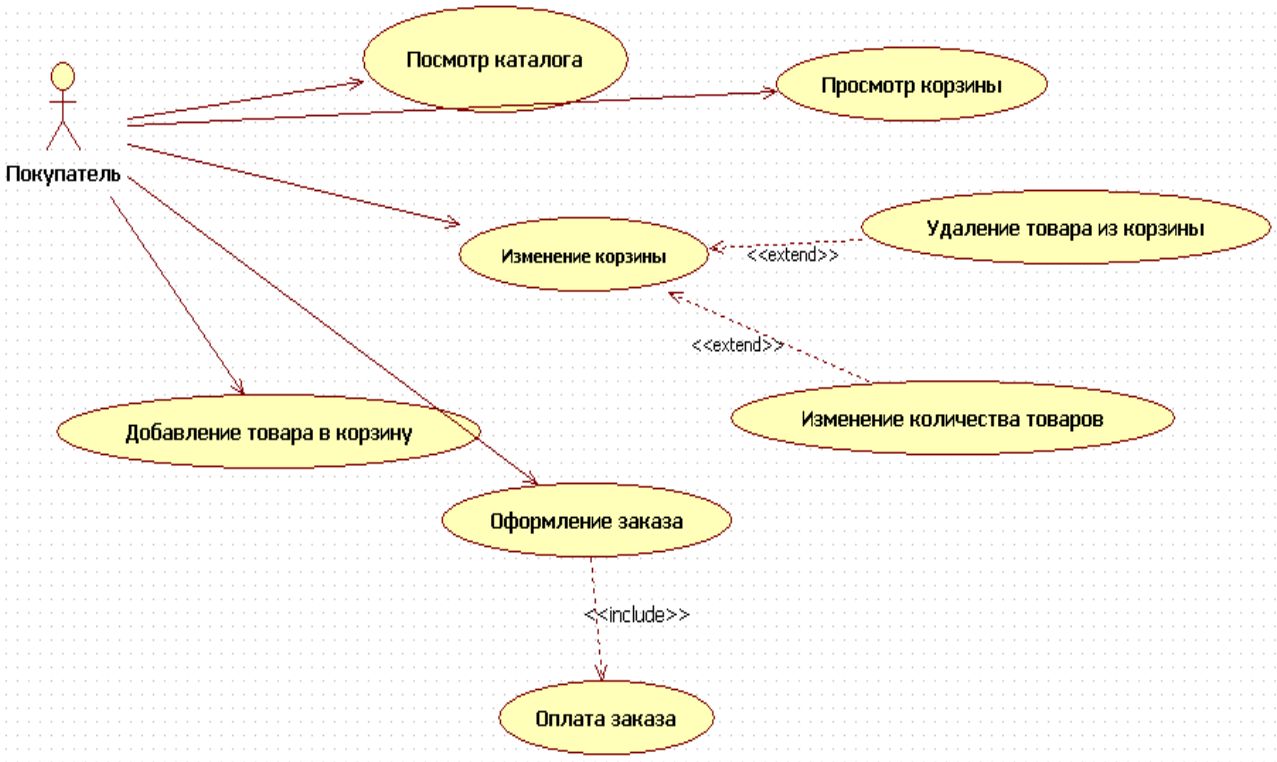


Рисунок 21. Диаграмма вариантов использования, поясняющая основной прецедент Заказ товаров

Потоки событий

Одним из требований языка UML является самодостаточность диаграмм для представления информации о моделях проектируемых систем. Однако, как уже отмечалось выше, диаграммы вариантов использования описывают то, что делает система, без уточнения того, как она это делает.

Для реального описания системы потребуются более специфические данные, которые отражены в потоке событий. Потоки событий уточняют или детализируют последовательность действий, совершаемых системой при выполнении ее вариантов использования, а также описывают логику переходов через варианты использования.

Поток событий – это определенная последовательность действий, которая описывает действия актеров и поведение моделируемой системы в форме обычного текста.

Потоки событий – это текстовые описания пошагового выполнения прецедентов, они понятны не только разработчику, но и стороннему читателю. Их задача – еще больше детализировать описание функциональности системы до того, как разработчики приступят к написанию программного кода, и устранить возможное недопонимание требуемой функциональности, как можно больше сблизить представления разработчика о системе и заказчика.

Потоки событий бывают трех типов**: основной, альтернативный и поток ошибок**.

Основной (главный) поток описывает наилучший сценарий либо наиболее используемый путь исполнения прецедента.

Альтернативный поток специфицирует отклонения от основного потока, которые не рассматриваются как ошибочные.

Поток ошибок рассматривается как отклонение от альтернативного или основного, которое порождает условия формирования ошибки.

Пример. *Потоки событий прецедента Заказ товаров.*

Основной поток событий.

1. Прецедент начинается с выбора покупателем режима показа каталога.
2. Система открывает каталог.
3. Покупатель выбирает режим показа корзины.

А1. Покупатель просматривает каталог и запускает поток «добавление товара в корзину»

4. Система открывает корзину.
5. Покупатель нажимает кнопку «оформить заказ».

А2. Покупатель просматривает корзину и запускает поток «изменение корзины».

А3. Покупатель решает вернуться в каталог.

6. Система переходит к первому шагу оформления заказа: запрашивает у покупателя личные данные и телефон.
7. Покупатель вводит личные данные и телефон.
8. Система переходит ко второму шагу оформления заказа: показывает содержимое заказа и просит подтвердить заказ.
9. Покупатель подтверждает заказ.

А4. Покупатель возвращается в корзину.

10. Система переходит к третьему шагу оформления заказа: запрашивает тип кредитной карты, ее номер, секретный код, имя владельца и дату завершения срока действия.
11. Покупатель вводит тип кредитной карты, ее номер, секретный код, имя владельца и дату завершения срока действия.

12. Система переходит к четвертому шагу оформления заказа: подтверждает оплату.

A5. Счет пользователя не найден.

A6. Недостаточно денег на счете.

E1. Платежная система недоступна.

13. Система присваивает заказу номер и отправляет его вместе с подтверждением заказа на электронный адрес покупателя.

14. Вариант использования завершается.

Альтернативные потоки.

A1. добавление товара в корзину

1. Покупатель выбирает размер.
2. Покупатель выбирает количество.
3. Покупатель нажимает кнопку Добавить в корзину
4. Система помещает выбранный товар в корзину.

A7. Покупатель не выбрал размер.

A8. Покупатель не выбрал количество.

5. Система выводит сообщение о том, что товар добавлен в корзину.
6. Поток возвращается на второй этап основного потока.

A2. Изменение корзины

1. Покупатель нажимает кнопку «Удалить» напротив одного выбранного товара.

A9. Покупатель изменяет количество позиций одного выбранного товара.

2. Система удаляет товар из корзины.
3. Поток возвращается к этапу 4 основного потока.

A3. Покупатель решает вернуться в каталог

1. Поток возвращается к этапу 2 основного потока.

A4. Покупатель возвращается в корзину

1. Поток возвращается к этапу 4 основного потока.

A5. Счет пользователя не найден

1. Система выводит сообщение о том, что счет пользователя не обнаружен.
2. Поток возвращается к этапу 11 основного потока.

A6. Недостаточно денег на счете

1. Система выводит сообщение о том, что на счете пользователя недостаточно денег для совершения операции.
2. Поток возвращается к этапу 11 основного потока.

A7. Покупатель не выбрал размер

1. Система выводит сообщение о необходимости выбора размера.
2. Поток возвращается на 1 этап потока A1.

A8. Покупатель не выбрал количество

1. Система выводит сообщение о необходимости выбора количества товара.
2. Поток возвращается на 2 этап потока A1.

A9. Покупатель изменяет количество позиций одного выбранного товара

1. Покупатель увеличивает количество товара.
2. Система обновляет корзину.

A10. Недостаточно товара в наличии.

3. Поток возвращается к этапу 4 основного потока.

A10. Недостаточно товара в наличии

1. Система выводит сообщение о том, сколько позиций может заказать покупатель.
2. Поток возвращается к этапу 4 основного потока.

Потоки ошибок.

E1. Платежная система недоступна

1. Система выводит сообщение о недоступности платежной системы.
2. Поток возвращается к этапу 11 основного потока.

Задание 2.6. Добавьте потоки событий к модели

Чтобы добавить поток событий к модели, нужно выделить прецедент, который он детализирует (в данном примере это прецедент Заказ товаров), в инспекторе модели открыть редактор вложений Attachments, нажать на значок , расположенный в верхней части редактора вложений, в появившемся окне нажать кнопку и выбрать соответствующий файл, содержащий описание потока событий. Чтобы удалить вложенный файл, выделите его на редакторе вложений и нажмите кнопку (рис. 22).

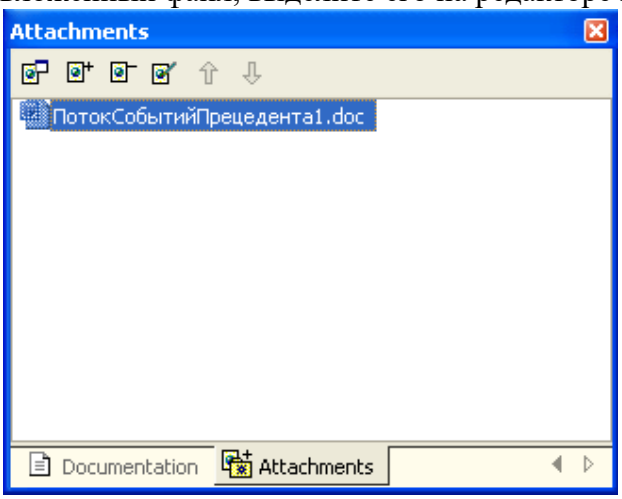


Рисунок 22. Добавление вложений в модель

Задание 3

Постройте диаграмму прецедентов

Секретарь размещает на сервере *меню* обеденных блюд на неделю. Сотрудники должны иметь возможность ознакомиться с *меню* и сделать заказ, выбрав блюда на каждый день следующей недели. *Офис*-менеджер должен иметь возможность сформировать счет и оплатить его. (Интернет-*приложение* для автоматизации заказов обедов в *офис*)

Таблица с описанием требований:

Прецедент	**Действующее лицо**
разместить меню	секретарь
ознакомиться с меню	сотрудник, секретарь, офис-менеджер
сделать заказ	сотрудник, секретарь, офис-менеджер
сформировать счет	офис-менеджер
оплатить счет	офис-менеджер

Для построения диаграмм воспользуйтесь CASE-средствами: StarUML

4. Создайте отчет по плану:

Оформить отчет по плану:

1) *Наименование работы*
2) *Цель*
3) *Задачи*
4) *Описание хода работы с отражением результата деятельности (**скриншоты диаграмм**)*
5) *Ответить на контрольные вопросы:*
    1. ***Основная структурная единица в StarUML.***
    2. ***Перечислите пакеты представлений в подходе Rational***
    3. ***Приведите примеры Actor.***
    4. ***Что такое вариант использования?***
    5. ***Поясните типы связей, которые можно использовать на диаграмме прецедентов.***